Experiment No.02

Aim: Implement Multi Regression, Lasso, and Ridge Regression on real-world datasets

Theory:

1. Dataset Source

Dataset: Insurance dataset
Source; Kaggle

Link: https://www.kaggle.com/datasets/noordeen/insurance-premium-prediction/data

2. Dataset Description

The Insurance Premium Prediction dataset contains information about individuals and the corresponding medical insurance costs billed by the insurance company.

```
data = pd.read_csv("sample_data/insurance.csv")

print(data.head())
print(data.info())
```

```
   age     sex   bmi  children smoker     region  expenses
0   19  female  27.9         0    yes  southwest  16884.92
1   18    male  33.8         1     no  southeast   1725.55
2   28    male  33.0         3     no  southeast   4449.46
3   33    male  22.7         0     no  northwest  21984.47
4   32    male  28.9         0     no  northwest   3866.86
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   expenses  1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
None
```

3. Mathematical Formulation of the Algorithm

Multiple Linear Regression :

Multiple Linear Regression models the relationship between a dependent variable and multiple independent variables.

General Equation:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_n X_n + \epsilon$$

Ridge Regression :

Ridge regression adds L2 regularization to reduce overfitting.

Objective Function:

$$\min_{\beta} \left(\sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2 \right)$$

Lasso Regression:

Lasso regression applies L1 regularization, which can perform feature selection.

Objective Function:

$$\min_{\beta} \left(\sum (y_i - \hat{y}_i)^2 + \lambda \sum |\beta_j| \right)$$

4. Algorithm Limitations

Multiple Linear Regression Limitations

- Sensitive to multicollinearity among features
- Performs poorly with non-linear relationships
- Highly influenced by outliers
- Requires assumptions such as normality and homoscedasticity

Ridge Regression Limitations

- Does not eliminate irrelevant features
- All coefficients remain non-zero

- Requires tuning of regularization parameter

Lasso Regression Limitations

- Can remove important variables if regularization is too strong
- Performance decreases when highly correlated features exist
- May produce unstable feature selection in some datasets

5. Methodology / Workflow

The following steps were followed to perform the experiment.

Step 1: Dataset Collection

The insurance dataset was downloaded from Kaggle.

Step 2: Data Loading

The dataset was loaded into Python using Pandas.

Step 3: Data Preprocessing

- Handling categorical variables using One Hot Encoding
- Feature scaling using StandardScaler
- Separating input features and target variable

Step 4: Data Splitting

Dataset was divided into:

- Training Set (80%)
- Testing Set (20%)

Step 5: Model Training

Three regression models were implemented:

- Multiple Linear Regression
- Ridge Regression
- Lasso Regression

Step 6: Model Evaluation

Models were evaluated using:

- Mean Squared Error (MSE)
- R^2 Score

Step 7: Visualization

Graphs were created to visualize:

- Actual vs Predicted values
- Model performance comparison using bar graphs

6. Performance Analysis

The performance of the models was evaluated using R^2 Score and Mean Squared Error.

Evaluation Metrics

Mean Squared Error (MSE)

$$MSE = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$$

Measures the average squared difference between predicted and actual values.

R^2 Score

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Measures how well the model explains the variance in the data.

- All three models performed similarly on the dataset.
- Ridge regression slightly improved generalization by controlling coefficient magnitudes.
- Lasso regression helped reduce the influence of less important features.
- The scatter plot of actual vs predicted values showed that predictions closely followed the true values.

7. Hyperparameter Tuning

Hyperparameter tuning was performed to optimize model performance.

Ridge Regression Hyperparameter:

Parameter tuned:

- alpha (λ) – Regularization strength.

Example values tested:

alpha = [0.01, 0.1, 1, 10, 100]

Lower alpha → weaker regularization
Higher alpha → stronger regularization

Optimal alpha was selected based on best R^2 score.

Lasso Regression Hyperparameter:

Parameter tuned:

- alpha

Example values tested:

alpha = [0.001, 0.01, 0.1, 1]

Higher alpha increases sparsity and removes less important features.

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
```

```
#Load Dataset
data = pd.read_csv("insurance.csv")
print(data.head())
print(data.info())

#Define Features and Target
X = data.drop("charges", axis=1)
y = data["charges"]

#Identify Categorical and Numerical Columns
categorical_features = ["sex", "smoker", "region"]
numerical_features = ["age", "bmi", "children"]


#Preprocessing

preprocessor = ColumnTransformer(

   transformers=[

       ("num", StandardScaler(), numerical_features),

       ("cat", OneHotEncoder(), categorical_features)

   ]

)

#Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(

   X, y, test_size=0.2, random_state=42

)

#Multiple Linear Regression

linear_model = Pipeline(steps=[

   ("preprocessor", preprocessor),

   ("model", LinearRegression())

])
```

```
linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)

print("Multiple Linear Regression")

print("MSE:", mean_squared_error(y_test, y_pred_linear))

print("R2 Score:", r2_score(y_test, y_pred_linear))

#Ridge Regression

ridge_model = Pipeline(steps=[

   ("preprocessor", preprocessor),

   ("model", Ridge(alpha=1.0))

])

ridge_model.fit(X_train, y_train)

y_pred_ridge = ridge_model.predict(X_test)

print("\nRidge Regression")

print("MSE:", mean_squared_error(y_test, y_pred_ridge))

print("R2 Score:", r2_score(y_test, y_pred_ridge))

#Lasso Regression

lasso_model = Pipeline(steps=[

   ("preprocessor", preprocessor),

   ("model", Lasso(alpha=0.1))

])

lasso_model.fit(X_train, y_train)

y_pred_lasso = lasso_model.predict(X_test)
```

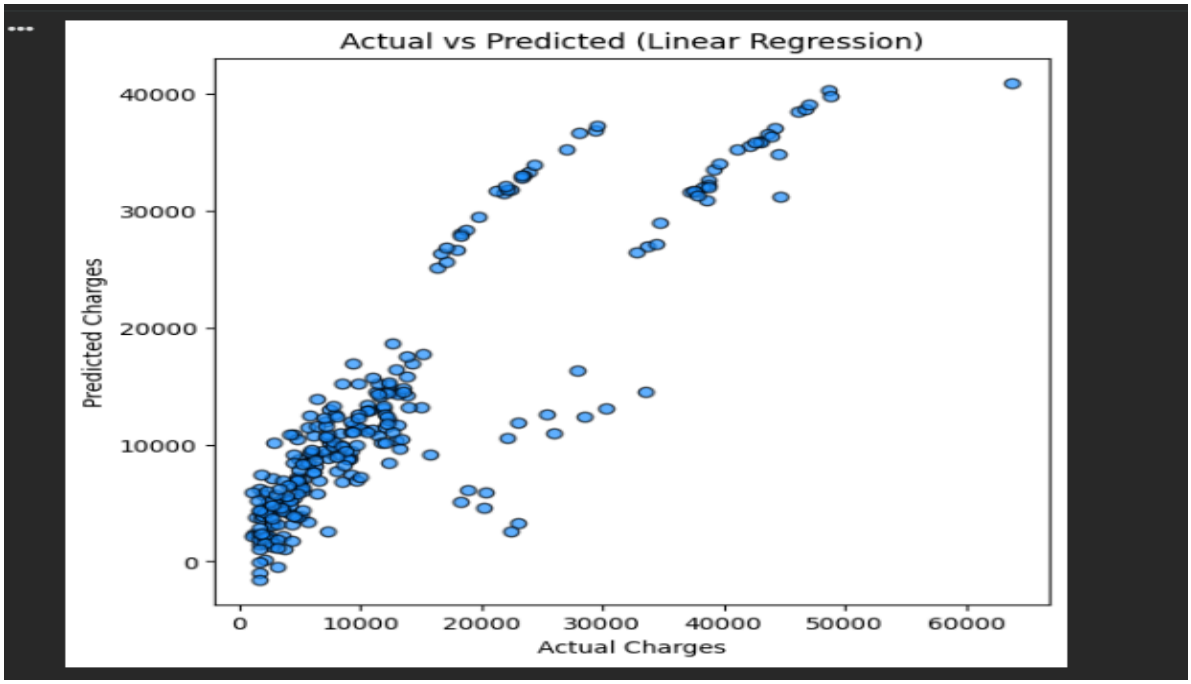
```
print("\nLasso Regression")

print("MSE:", mean_squared_error(y_test, y_pred_lasso))

print("R2 Score:", r2_score(y_test, y_pred_lasso))


#Visualization (Actual vs Predicted)

plt.figure(figsize=(6,6))

plt.scatter(y_test, y_pred_linear)

plt.xlabel("Actual Charges")

plt.ylabel("Predicted Charges")

plt.title("Actual vs Predicted (Linear Regression)")

plt.show()
```

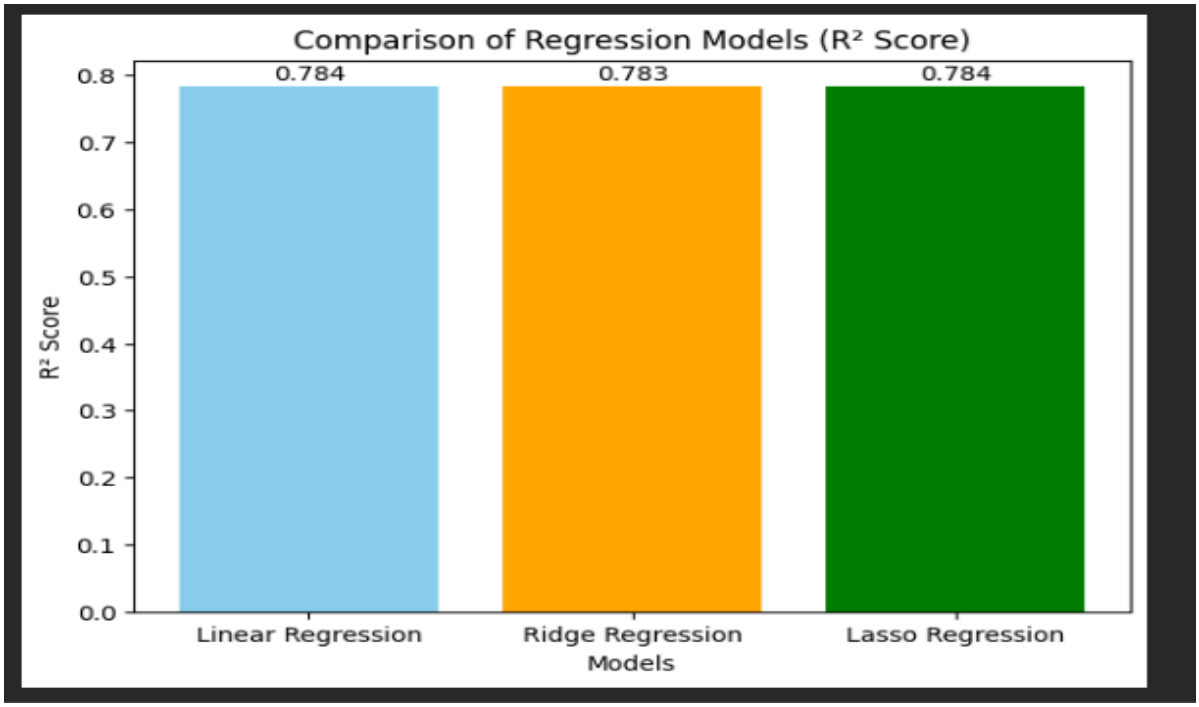
Output:

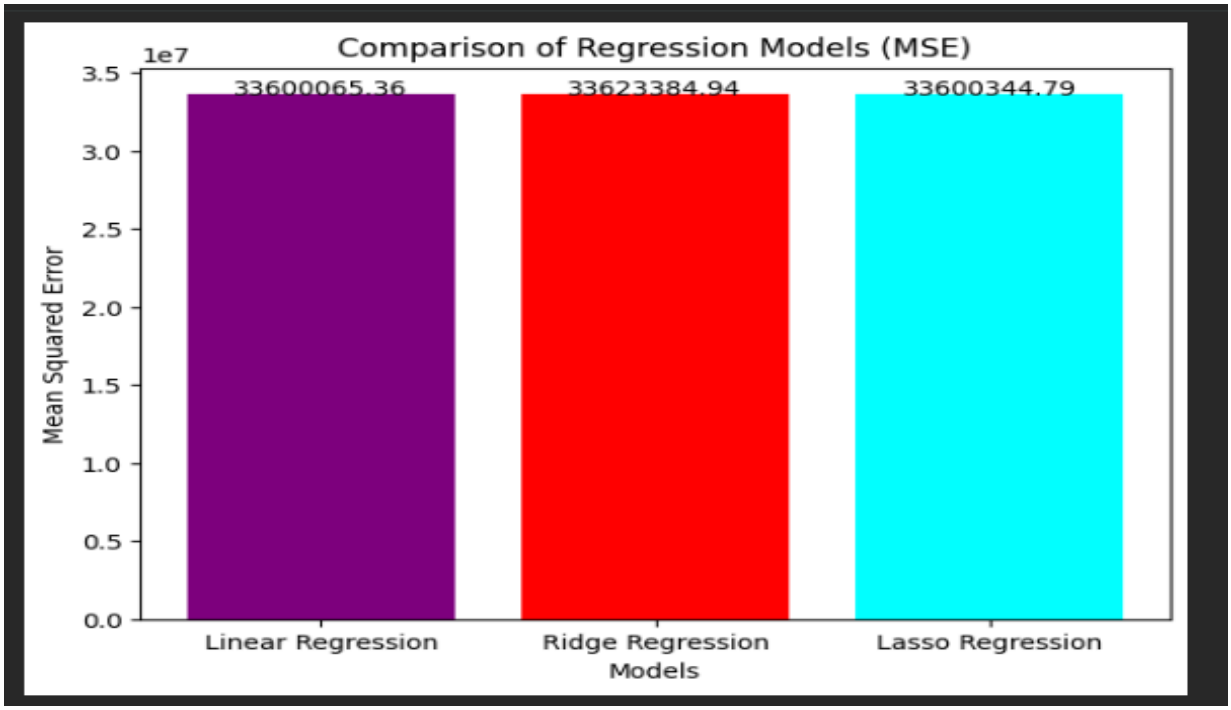
```
Multiple Linear Regression
MSE: 33600065.35507783
R2 Score: 0.7835726930039905
```

```
Ridge Regression
MSE: 33623384.93692298
R2 Score: 0.7834224851324973
```

```
Lasso Regression
MSE: 33600344.79240548
R2 Score: 0.7835708930709351
```


Actual vs Predicted (Linear Regression)
Predicted Charges
Actual Charges
0
10000
20000
30000
40000
50000
60000


Comparison of Regression Models (R² Score)
R² Score
0.784
0.783
0.784
Linear Regression
Ridge Regression
Lasso Regression
Models



Conclusion:

In this experiment, Multiple Linear Regression, Ridge Regression, and Lasso Regression were implemented on the Insurance Premium dataset. The models successfully predicted insurance charges based on demographic and health-related features. Ridge regression helped control overfitting, while Lasso regression performed implicit feature selection. Performance evaluation showed that regularization techniques can improve model stability and prediction reliability.